Créer l'application Moutures

Cours pour débutants — Kotlin et Jetpack Compose

Dans ce cours, vous allez comprendre, pas à pas, comment fonctionne l'application **Moutures** : une application Android qui lit un fichier JSON au démarrage, affiche la liste des moutures et montre le détail de celle que vous touchez, sur le même écran, au-dessus de la liste.

Ce que vous saurez faire à la fin :

- lire un fichier JSON placé dans les assets ;
- ranger des données dans des **data class** Kotlin ;
- garder l'état de l'écran dans un **ViewModel** ;
- construire un écran avec **Column**, **Row**, **LazyColumn**, **Card** ;
- faire un tableau, afficher des icônes, arrondir des bordures ;
- personnaliser les couleurs avec le **thème Material 3** ;
- comprendre et corriger les erreurs de compilation les plus courantes.

Prérequis : aucun en Compose. Savoir ce qu'est une variable et une fonction aide, mais chaque notion Kotlin utilisée est expliquée.

Sommaire

1. Ce que fait l'application 3
2. Le fichier de données JSON 4
3. Le modèle de données 5
4. Lire le fichier JSON 6
5. Le ViewModel et l'état de l'écran 7
6. Les bases de Jetpack Compose 9
7. L'écran principal 11
8. Les lignes et la carte de détail 13
9. Tableau, icônes et bordures 15
10. Couleurs et thème 17
11. Bibliothèques et lancement 19
12. Erreurs fréquentes et solutions 20
13. Exercices 21
14. Glossaire 22

1. Ce que fait l'application

L'application affiche des **moutures** (t55, Tradition, Originale…) et leurs réglages. Elle fonctionne en quatre étapes :

Étape	Ce qui se passe	Où dans le code
1. Démarrage	Android lance l'activité principale.	`MainActivity`
2. Lecture	Le fichier **rec.json** est lu et transformé en objets Kotlin.	`MoutureRepository`
3. État	La liste et la mouture sélectionnée sont gardées en mémoire.	`MoutureViewModel`
4. Affichage	L'écran dessine la carte de détail puis la liste.	`MouturesScreen`

À l'écran, de haut en bas :

- une **barre de titre** « Moutures » ;
- une **carte de détail** de la mouture sélectionnée : nom, tableau des vitesses, gruau, améliorant, mignon, remoulage, dév. sûreté ;
- la **liste** de toutes les moutures. Toucher une ligne change la carte de détail.

Au lancement, la première mouture est déjà sélectionnée : la carte est visible tout de suite.

Les fichiers du projet

```
app/
├── build.gradle.kts            ← les bibliothèques utilisées
└── src/main/
    ├── AndroidManifest.xml     ← déclare l'application et son activité
    ├── assets/
    │   └── rec.json            ← les données des moutures
    └── kotlin/com/example/moutures2/
        ├── MainActivity.kt     ← point d'entrée
        ├── Mouture.kt          ← le modèle de données
        ├── MoutureRepository.kt← lecture du JSON
        ├── MoutureViewModel.kt ← l'état de l'écran
        ├── MouturesScreen.kt   ← l'interface (Compose)
        └── Theme.kt            ← les couleurs
```

Bon à savoir
Le dossier peut s'appeler **java** ou **kotlin** selon votre éditeur : les deux fonctionnent. Ce qui compte, c'est que la ligne `package` en haut de chaque fichier corresponde au nom du dossier.

2. Le fichier de données JSON

JSON est un format texte pour décrire des données. Il utilise deux structures :

- l'**objet**, entre accolades `{ }` : des paires clé : valeur ;
- le **tableau**, entre crochets `[ ]` : une liste de valeurs.

Voici un extrait de **rec.json** :

assets/rec.json

```
{
  "Tradition": {
    "sm": { "vb": 21, "vd": 72 },
    "am": { "vb": 19, "vd": 67 },
    "info": {
      "gruau": "doseur à 18",
      "ameliorant": "Floricreme",
      "mignon": "doseur à 18",
      "remoulage": [true, false, false],
      "dev_surete": false
    }
  },
  "t55": { ... }
}
```

Clé	Signification	Type
`Tradition`	Le nom de la mouture (c'est la clé elle-même)	texte
`sm` / `am`	Réglages **Sans Meule** / **Avec Meule**	objet
`vb` / `vd`	**Vit**esse **B**lé / **Vit**esse **D**oseur	nombre
`gruau, ameliorant, mignon`	Texte, ou `null` si rien	texte ou null
`remoulage`	3 cases cochées ou non	tableau de booléens
`dev_surete`	Oui / non	booléen

Pourquoi dans assets ?

Le dossier `src/main/assets` est copié tel quel dans l'application installée. Le code peut ensuite ouvrir le fichier par son nom avec `context.assets.open("rec.json")`.

Piège fréquent
Une seule virgule en trop ou un guillemet oublié, et la lecture échoue. L'application affiche alors le message d'erreur prévu (« Impossible de lire rec.json »). En cas de doute, collez votre JSON dans un validateur en ligne.

3. Le modèle de données

Avant de lire le JSON, on décrit la **forme** des données en Kotlin. On utilise des **data class** : des classes faites pour contenir des valeurs.

Mouture.kt

```
package com.example.moutures2

data class Dosage(
    val vb: Int,   // vitesse blé
    val vd: Int,   // vitesse doseur
)

data class InfoMouture(
    val gruau: String?,
    val ameliorant: String?,
    val mignon: String?,
    val remoulage: List<Boolean>,
    val devSurete: Boolean,
)

data class Mouture(
    val nom: String,
    val sm: Dosage,        // Sans Meule
    val am: Dosage,        // Avec Meule
    val info: InfoMouture,
)
```

Les mots à connaître

Écriture	Signification
`val`	Valeur qui ne change plus une fois donnée (lecture seule).
`Int`, `String`, `Boolean`	Nombre entier, texte, vrai/faux.
`String?`	Le **?** veut dire « peut être **null** » (absent). Indispensable ici, car `gruau` vaut `null` pour t55.
`List`	Une liste de vrai/faux, comme `[true, false, false]`.
`data class`	Kotlin génère automatiquement l'égalité, l'affichage, et `copy()` qui sert beaucoup avec l'état.

À retenir
Une `Mouture` contient deux `Dosage` et une `InfoMouture`. Le code suit exactement l'imbrication du JSON.

4. Lire le fichier JSON

Le **repository** (« dépôt ») a un seul rôle : aller chercher les données et les transformer en objets `Mouture`. On utilise `org.json`, inclus dans Android : aucune bibliothèque à ajouter.

MoutureRepository.kt (extrait)

```
class MoutureRepository(private val context: Context) {

    suspend fun chargerMoutures(): List<Mouture> = withContext(Dispatchers.IO) {
        val texte = context.assets.open("rec.json")
            .bufferedReader(Charsets.UTF_8)
            .use { it.readText() }
        parser(texte)
    }

    companion object {
        fun parser(json: String): List<Mouture> {
            val racine = JSONObject(json)
            return racine.keys().asSequence().map { nom ->
                val obj = racine.getJSONObject(nom)
                Mouture(
                    nom = nom,
                    sm = obj.optJSONObject("sm").toDosage(),
                    am = obj.optJSONObject("am").toDosage(),
                    info = obj.optJSONObject("info").toInfo(),
                )
            }.toList()
        }
        // toDosage() et toInfo() lisent chaque champ avec optInt, optBoolean...
    }
}
```

Ligne par ligne

- `assets.open("rec.json")` ouvre le fichier ; `readText()` le lit en entier ; `.use { }` le referme automatiquement.
- `JSONObject(json)` transforme le texte en objet JSON manipulable.
- `racine.keys()` donne les noms : t55, Tradition, Originale. Chaque nom devient une `Mouture`.
- `optInt`**,** `optBoolean` : le préfixe opt veut dire « optionnel » : si la clé manque, on obtient une valeur par défaut au lieu d'un plantage.

suspend et Dispatchers.IO

Lire un fichier prend du temps. Si on le fait sur le fil principal (celui qui dessine l'écran), l'application peut se figer. `withContext(Dispatchers.IO)` exécute la lecture sur un fil dédié aux entrées/sorties. Le mot `suspend` indique que la fonction peut attendre sans bloquer : elle doit être appelée depuis une **coroutine** (on le fait dans le ViewModel).

À retenir
Le repository ne sait rien de l'écran. Il lit, il convertit, il renvoie une liste.

5. Le ViewModel et l'état de l'écran

L'**état**, c'est tout ce que l'écran doit afficher à un instant donné. On le regroupe dans une seule data class :

MoutureViewModel.kt

```
data class MouturesUiState(
    val chargement: Boolean = true,
    val moutures: List<Mouture> = emptyList(),
    val selection: Mouture? = null,
    val erreur: String? = null,
)
```

Le **ViewModel** garde cet état et le fait évoluer. Gros avantage : il **survit à la rotation de l'écran**. Sans lui, tourner le téléphone relirait le JSON et perdrait la sélection.

```
class MoutureViewModel(application: Application) : AndroidViewModel(application) {

    private val repository = MoutureRepository(application)

    private val _uiState = MutableStateFlow(MouturesUiState())
    val uiState: StateFlow<MouturesUiState> = _uiState.asStateFlow()

    init {
        // Lecture du JSON au démarrage, puis sélection du 1er élément.
        viewModelScope.launch {
            _uiState.value = try {
                val moutures = repository.chargerMoutures()
                MouturesUiState(
                    chargement = false,
                    moutures = moutures,
                    selection = moutures.firstOrNull(),
                )
            } catch (e: Exception) {
                MouturesUiState(chargement = false,
                    erreur = "Impossible de lire rec.json : ${e.message}")
            }
        }
    }

    fun onMoutureClick(mouture: Mouture) {
        _uiState.update { it.copy(selection = mouture) }
    }
}
```

Les notions clés

Élément	Rôle
`MutableStateFlow`	Une « boîte » qui contient l'état et prévient quand il change. Privée (`_uiState`) : seul le ViewModel peut la modifier.
`StateFlow`	La version en lecture seule, exposée à l'écran.
`init { }`	Code exécuté à la création du ViewModel, donc au démarrage.
`viewModelScope.launch`	Lance une coroutine, arrêtée automatiquement si le ViewModel disparaît.
`firstOrNull()`	Le premier élément, ou `null` si la liste est vide (pas de plantage).
`it.copy(selection = …)`	Crée une copie de l'état avec seulement la sélection changée.

Bon à savoir
Variante « bascule » : pour qu'un second toucher sur la même mouture referme la carte, écrivez `selection = if (it.selection?.nom == mouture.nom) null else mouture`.

À retenir
L'écran ne modifie jamais l'état lui-même : il **appelle** une fonction du ViewModel (`onMoutureClick`), qui met l'état à jour, et l'écran se redessine.

6. Les bases de Jetpack Compose

Avec Compose, on décrit l'interface avec des **fonctions Kotlin** marquées `@Composable`. Quand l'état change, Compose rappelle ces fonctions et met l'écran à jour : c'est la **recomposition**.

```
@Composable
fun Bonjour(nom: String) {
    Text(text = "Bonjour $nom")
}
```

Piège fréquent
Une fonction `@Composable` ne peut être appelée que depuis une autre fonction `@Composable`. Sinon : « @Composable invocations can only happen from the context of a @Composable function ».

Les trois conteneurs de base

Conteneur	Dispose ses enfants…	Exemple dans l'appli
`Column`	les uns **sous** les autres	Le contenu de la carte de détail
`Row`	les uns **à côté** des autres	Libellé à gauche, valeur à droite
`Box`	les uns **par-dessus** les autres	Centrer l'indicateur de chargement

Le Modifier

Le `Modifier` règle la taille, les marges, le fond, la bordure, le clic… On enchaîne les réglages avec des points :

```
Modifier
    .fillMaxWidth()                    // toute la largeur
    .background(Color.Yellow)          // fond jaune
    .padding(16.dp)                    // marge intérieure
    .clickable { /* action */ }        // réagit au toucher
```

Piège fréquent
L'ordre compte ! `.background().padding()` : le fond couvre aussi la marge.
`.padding().background()` : la marge reste transparente, autour du fond.

Les unités

- **dp** (density-independent pixel) pour les tailles et les marges : même taille physique sur tous les écrans.
- **sp** pour le texte : comme dp, mais suit aussi la taille de police choisie par l'utilisateur dans les réglages d'Android.

weight : partager la place

`Modifier.weight(1f)` donne à un élément une part de la place restante. Il n'existe **que** dans une `Row` ou une `Column`.

```
Row(Modifier.fillMaxWidth()) {
    Text("Gruau", Modifier.weight(1f))   // prend toute la place libre
    Text("doseur à 18")                  // juste sa taille, collé à droite
}
```

7. L'écran principal

Relier l'écran au ViewModel

MouturesScreen.kt

```
@Composable
fun MouturesScreen(viewModel: MoutureViewModel = viewModel()) {
    val etat by viewModel.uiState.collectAsStateWithLifecycle()
    MouturesContent(etat = etat, onMoutureClick = viewModel::onMoutureClick)
}
```

- `viewModel()` récupère le ViewModel (le même après une rotation). Il vient de la bibliothèque `lifecycle-viewmodel-compose`.
- `collectAsStateWithLifecycle()` transforme le `StateFlow` en état Compose : chaque changement provoque une recomposition. Bibliothèque `lifecycle-runtime-compose`.
- `by` permet d'écrire `etat` au lieu de `etat.value`.
- `viewModel::onMoutureClick` passe la **fonction** elle-même, sans l'appeler.

Bon à savoir
Séparer `MouturesScreen` (qui connaît le ViewModel) et `MouturesContent` (qui ne reçoit que des données) permet d'afficher `MouturesContent` dans un aperçu `@Preview` sans lancer l'application.

Scaffold et when

```
Scaffold(topBar = { TopAppBar(title = { Text("Moutures") }) }) { padding ->
    Box(Modifier.fillMaxSize().padding(padding)) {
        when {
            etat.chargement -> CircularProgressIndicator(Modifier.align(Alignment.Center))
            etat.erreur != null -> Text(etat.erreur, color = MaterialTheme.colorScheme.error)
            else -> LazyColumn( /* ... voir page suivante ... */ )
        }
    }
}
```

- **Scaffold** fournit la structure : barre du haut, contenu. Son `padding` indique la place prise par les barres : on l'applique au contenu pour ne rien cacher.
- **when** choisit quoi afficher : la roue de chargement, l'erreur, ou la liste. La première condition vraie gagne.

La LazyColumn

Une `LazyColumn` est une liste qui défile et ne dessine que les éléments visibles. Dedans, on n'écrit pas des composables directement : on déclare des **éléments** avec `item { }` (un seul) et `items(liste) { }` (plusieurs).

```
val listState = rememberLazyListState()

LaunchedEffect(etat.selection) {
    if (etat.selection != null) listState.animateScrollToItem(0)
}

LazyColumn(
    state = listState,
    modifier = Modifier.fillMaxSize(),
    contentPadding = PaddingValues(horizontal = 16.dp, vertical = 8.dp),
    verticalArrangement = Arrangement.spacedBy(8.dp),
) {
    // 1) la carte de détail, en tête de liste
    etat.selection?.let { mouture ->
        item(key = "detail") { DetailMouture(mouture) }
    }
    // 2) une ligne par mouture
    items(etat.moutures, key = { it.nom }) { mouture ->
        LigneMouture(
            mouture = mouture,
            selectionnee = mouture.nom == etat.selection?.nom,
            onClick = { onMoutureClick(mouture) },
        )
    }
}
```

- `etat.selection?.let { }` : le bloc ne s'exécute que si une mouture est sélectionnée.
- `key` : un identifiant unique par élément ; Compose s'y retrouve quand la liste change.
- `rememberLazyListState()` retient la position de défilement, même après rotation.
- `LaunchedEffect(etat.selection)` s'exécute à chaque changement de sélection : la liste remonte en haut pour montrer la carte.

Pourquoi la carte est DANS la liste ?

En mode paysage, l'écran est bas. Une carte placée au-dessus de la liste prendrait toute la hauteur. Dans la liste, la carte et les lignes défilent ensemble.

Piège fréquent
Ne mettez pas une `LazyColumn` dans une `Column` avec `verticalScroll` : l'application plante (« measured with an infinity maximum height constraints »). Deux listes qui défilent dans le même sens ne s'emboîtent pas.

8. Les lignes et la carte de détail

Une ligne de la liste

```
@Composable
private fun LigneMouture(mouture: Mouture, selectionnee: Boolean, onClick: () -> Unit) {
    Card(
        modifier = Modifier.fillMaxWidth().clickable(onClick = onClick),
        colors = CardDefaults.cardColors(
            containerColor = if (selectionnee) MaterialTheme.colorScheme.primaryContainer
                             else MaterialTheme.colorScheme.surfaceVariant,
        ),
    ) {
        Row(Modifier.padding(16.dp), verticalAlignment = Alignment.CenterVertically) {
            Text(mouture.nom, style = MaterialTheme.typography.titleMedium,
                 modifier = Modifier.weight(1f))
            Text("SM ${mouture.sm.vb}/${mouture.sm.vd} · AM ${mouture.am.vb}/${mouture.am.vd}",
                 style = MaterialTheme.typography.bodySmall)
        }
    }
}
```

- `onClick: () -> Unit` est un paramètre de type **fonction** : la ligne ne sait pas ce qui se passe au clic, elle prévient simplement celui qui l'a créée.
- La couleur de fond change selon `selectionnee` : la ligne choisie est mise en valeur.
- `"${mouture.nom}"` : le `$` insère une valeur dans un texte.

La carte de détail

```
@Composable
private fun DetailMouture(mouture: Mouture) {
    Card(
        modifier = Modifier.fillMaxWidth().padding(bottom = 8.dp),
        colors = CardDefaults.cardColors(
            containerColor = MaterialTheme.colorScheme.secondaryContainer),
    ) {
        Column(Modifier.padding(16.dp), verticalArrangement = Arrangement.spacedBy(6.dp)) {
            Text(mouture.nom, style = MaterialTheme.typography.headlineSmall,
                 fontWeight = FontWeight.Bold)
            TableauDosage(sm = mouture.sm, am = mouture.am)       // chapitre 9
            HorizontalDivider(Modifier.padding(vertical = 4.dp))
            val info = mouture.info
            LigneInfo("Gruau", info.gruau ?: "—")
            LigneInfo("Améliorant", info.ameliorant ?: "—")
            LigneInfo("Mignon", info.mignon ?: "—")
            IconesRemoulage(titre = "Remoulage", valeurs = info.remoulage)  // chapitre 9
            LigneInfo("Dév. sûreté", if (info.devSurete) "Oui" else "Non")
        }
    }
}
```

`info.gruau ?: "—"` : l'opérateur **Elvis** `?:` donne une valeur de remplacement quand la valeur est `null`.

```
@Composable
private fun LigneInfo(libelle: String, valeur: String) {
    Row(Modifier.fillMaxWidth()) {
        Text(libelle, style = MaterialTheme.typography.bodyMedium, modifier = Modifier.weight(1f))
        Text(valeur, style = MaterialTheme.typography.bodyMedium, fontWeight = FontWeight.Medium)
    }
}
```

Piège fréquent
Le tiret affiché pour une valeur vide est un tiret long « — ». Un test comme `valeur.contains('-')` cherche le tiret court du clavier et ne le trouvera jamais. Comparez plutôt avec le même texte : `valeur == "—"`, idéalement via une constante.

9. Tableau, icônes et bordures

Construire un tableau

Un tableau = une `Column` de lignes (`Row`). Toutes les cases d'une même colonne reçoivent le même `weight` : les colonnes restent alignées.

```
@Composable
private fun TableauDosage(sm: Dosage, am: Dosage) {
    Column(Modifier.fillMaxWidth().border(1.dp, Color.Gray)) {
        LigneTableau("", "Sans Meule", "Avec Meule", entete = true)
        HorizontalDivider(thickness = 1.dp, color = Color.Gray)
        LigneTableau("Vit Blé", sm.vb.toString(), am.vb.toString())
        HorizontalDivider(thickness = 1.dp, color = Color.Gray)
        LigneTableau("Vit Doseur", sm.vd.toString(), am.vd.toString())
    }
}

@Composable
private fun LigneTableau(libelle: String, a: String, b: String, entete: Boolean = false) {
    Row(Modifier.fillMaxWidth().height(IntrinsicSize.Min)) {
        CaseTableau(libelle, Modifier.weight(1.2f), gras = true, alignement = TextAlign.Start)
        VerticalDivider(thickness = 1.dp, color = Color.Gray)
        CaseTableau(a, Modifier.weight(1f), gras = entete)
        VerticalDivider(thickness = 1.dp, color = Color.Gray)
        CaseTableau(b, Modifier.weight(1f), gras = entete)
    }
}

@Composable
private fun CaseTableau(texte: String, modifier: Modifier = Modifier,
                        gras: Boolean = false, alignement: TextAlign = TextAlign.Center) {
    Text(texte, modifier = modifier.padding(horizontal = 8.dp, vertical = 6.dp),
         fontWeight = if (gras) FontWeight.SemiBold else FontWeight.Normal,
         textAlign = alignement)
}
```

Piège fréquent
Sans `.height(IntrinsicSize.Min)` sur la `Row`, les `VerticalDivider` sont invisibles : ils prennent la hauteur du parent, et la `Row` ne la connaît pas à l'avance. `IntrinsicSize.Min` lui donne la hauteur de son plus grand enfant.

Des icônes

```
@Composable
fun IconesRemoulage(titre: String, valeurs: List<Boolean>, modifier: Modifier = Modifier) {
    Column(modifier, verticalArrangement = Arrangement.spacedBy(4.dp)) {
        Text(titre, style = MaterialTheme.typography.titleSmall, fontWeight = FontWeight.SemiBold)
        Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
            List(3) { i -> valeurs.getOrElse(i) { false } }.forEachIndexed { i, actif ->
                Icon(
                    imageVector = if (actif) Icons.Filled.CheckCircle else Icons.Filled.Close,
                    contentDescription = "Remoulage ${i + 1} : ${if (actif) "oui" else "non"}",
                    tint = if (actif) Color(0xFF2E7D32) else MaterialTheme.colorScheme.error,
                    modifier = Modifier.size(28.dp),
                )
            }
        }
    }
}
```

- `contentDescription` est lu à voix haute par TalkBack pour les personnes malvoyantes.

- `getOrElse(i) { false }` évite un plantage s'il y a moins de 3 valeurs dans le JSON.

Piège fréquent
Importez `androidx.compose.material.icons.Icons` (sans « 3 »), et non `androidx.compose.material3.internal.Icons` que l'éditeur propose parfois : il est interne (« it is internal »). Ajoutez au besoin la bibliothèque `androidx.compose.material:material-icons-core`.

Arrondir une bordure

On passe une forme à `border`. Avec un fond, donnez-lui la **même** forme, et placez-le avant la bordure :

```
Modifier
    .fillMaxWidth()
    .background(MaterialTheme.colorScheme.surface, RoundedCornerShape(8.dp))
    .border(1.dp, Color.Black, RoundedCornerShape(8.dp))
    .padding(8.dp)
```

10. Couleurs et thème

Material 3 regroupe les couleurs dans un `ColorScheme`. Chaque couleur de fond a sa couleur de texte, qui porte le même nom précédé de **on**.

Fond	Texte dessus	Utilisé pour
`surface`	`onSurface`	Texte normal, barre du haut
`surfaceVariant`	`onSurfaceVariant`	Lignes non sélectionnées
`primaryContainer`	`onPrimaryContainer`	Ligne sélectionnée
`secondaryContainer`	`onSecondaryContainer`	Carte de détail
`primary`	`onPrimary`	Boutons, roue de chargement
`error`	`onError`	Messages et valeurs en erreur

Un `Card` ou une `Surface` choisit automatiquement la bonne couleur on… pour son texte : souvent, vous n'avez rien à préciser.

Theme.kt

```
private val LightColors = lightColorScheme(
    primary = Color(0xFF8D6E43),
    primaryContainer = Color(0xFFF3E2C7),
    secondaryContainer = Color(0xFFFBEBC4),
    onSecondaryContainer = Color(0xFF3E2C10),
)

@Composable
fun MouturesTheme(
    darkTheme: Boolean = isSystemInDarkTheme(),
    dynamicColor: Boolean = false,          // false = mes couleurs
    content: @Composable () -> Unit,
) {
    val colors = when {
        dynamicColor && Build.VERSION.SDK_INT >= Build.VERSION_CODES.S -> {
            val ctx = LocalContext.current
            if (darkTheme) dynamicDarkColorScheme(ctx) else dynamicLightColorScheme(ctx)
        }
        darkTheme -> DarkColors
        else -> LightColors
    }
    MaterialTheme(colorScheme = colors, content = content)
}
```

Piège fréquent
Sur Android 12 et plus, les **couleurs dynamiques** (tirées du fond d'écran) remplacent vos couleurs si la condition `dynamicColor &&` est absente. Si vos couleurs « ne marchent pas », c'est presque toujours ça.

Hauteur d'une ligne de texte

Style	Taille	Hauteur de ligne
`bodySmall`	12 sp	16 dp
`bodyMedium`	14 sp	20 dp
`bodyLarge` / `titleMedium`	16 sp	24 dp
`titleLarge`	22 sp	28 dp
`headlineSmall`	24 sp	32 dp

Valeurs avec la police à 100 %. Si l'utilisateur agrandit la police, le texte grandit : évitez de fixer en dp la hauteur d'un élément qui contient du texte.

11. Bibliothèques et lancement

app/build.gradle.kts (extrait)

```
dependencies {
    implementation(platform("androidx.compose:compose-bom:2024.12.01"))
    implementation("androidx.compose.ui:ui")
    implementation("androidx.compose.material3:material3")
    implementation("androidx.compose.material:material-icons-core")
    implementation("androidx.activity:activity-compose:1.9.3")
    implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.8.7")
    implementation("androidx.lifecycle:lifecycle-runtime-compose:2.8.7")
}
```

- Le **BOM** (`platform(...)`) fixe d'un coup des versions compatibles pour toutes les bibliothèques Compose : on n'écrit plus leur version.
- Après chaque modification de ce fichier : **Sync** (synchroniser).
- Gardez la même version pour les deux bibliothèques `lifecycle`.

MainActivity.kt

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            MouturesTheme { MouturesScreen() }
        }
    }
}
```

- `setContent { }` remplace l'ancien fichier de mise en page XML : tout l'écran est en Compose.
- `enableEdgeToEdge()` dessine l'application sous les barres du système. Le `Scaffold` et son `padding` décalent le contenu pour qu'il ne soit pas caché.

Bon à savoir
L'avertissement « targetSdk 36 enables edge-to-edge by default » peut être ignoré : `enableEdgeToEdge()` et le `Scaffold` gèrent déjà cette situation.

12. Erreurs fréquentes et solutions

Toutes ces erreurs ont été rencontrées en construisant cette application.

Message du compilateur	Cause	Solution
Expression 'weight' of type 'Float' cannot be invoked as a function	`weight` utilisé hors d'une `Row`/`Column` (par ex. dans un `Box`).	Remplacer par `Modifier.fillMaxSize()` ou entourer d'une `Column`.
@Composable invocations can only happen from the context of a @Composable function	Composable appelé directement dans le bloc d'une `LazyColumn`.	L'entourer de `item { … }`.
Cannot access 'object Icons': it is internal	Mauvais import : `material3.internal.Icons`.	Importer `androidx.compose.material.icons.Icons`.
Unresolved reference: Icons / viewModel	Bibliothèque manquante.	Ajouter `material-icons-core` ou `lifecycle-viewmodel-compose`, puis Sync.
measured with an infinity maximum height constraints (plantage)	`LazyColumn` dans un conteneur avec `verticalScroll`.	Mettre le contenu dans la `LazyColumn` avec `item { }`.
VerticalDivider invisible (pas d'erreur)	La `Row` n'a pas de hauteur connue.	Ajouter `.height(IntrinsicSize.Min)` à la `Row`.
Mes couleurs ne s'appliquent pas (pas d'erreur)	Couleurs dynamiques actives sur Android 12+.	Paramètre `dynamicColor = false` dans le thème.
Expecting '}' / accolade en trop	Bloc mis en commentaire sans son accolade fermante.	Compter les accolades ; l'éditeur surligne la paire.

13. Exercices

Faites-les dans l'ordre : chacun utilise une notion du cours. Les indices sont en bas.

Exercice 1 — Ajouter une mouture
Ajoutez une mouture « Seigle » dans **rec.json** avec vos propres valeurs. Relancez : elle doit apparaître en fin de liste, sans toucher au code Kotlin.

Exercice 2 — Changer un libellé
Remplacez « Dév. sûreté » par « Développement sûreté » dans la carte de détail.

Exercice 3 — Compteur
Affichez sous la barre de titre le texte « 3 moutures » (le nombre réel).

Exercice 4 — Valeur vide en couleur
Dans `LigneInfo`, affichez le « — » sur fond `error` avec le texte en `onError`.

Exercice 5 — Bascule
Faites qu'un second toucher sur la mouture sélectionnée referme la carte.

Exercice 6 — Ligne de tableau
Ajoutez au tableau une ligne « Total » qui affiche `vb + vd`.

Exercice 7 — Recherche (défi)
Ajoutez un `OutlinedTextField` au-dessus de la liste pour filtrer les moutures par nom.

Indices

1. Une nouvelle clé au même niveau que « Tradition », sans oublier la virgule entre deux moutures.
2. Cherchez `"Dév. sûreté"` dans `DetailMouture`.
3. Un `item { Text("${etat.moutures.size} moutures") }` en tout premier dans la `LazyColumn`.
4. `val vide = valeur == "—"`, puis `Modifier.background(...)` et `color = ...` selon `vide`.
5. Voir l'encadré « Variante bascule » du chapitre 5.
6. Un `HorizontalDivider` puis `LigneTableau("Total", (sm.vb + sm.vd).toString(), …)`.
7. `var texte by remember { mutableStateOf("") }`, puis `etat.moutures.filter { it.nom.contains(texte, ignoreCase = true) }`.

14. Glossaire

Terme	Définition
Activity	Un écran Android. Ici une seule : `MainActivity`.
assets	Dossier de fichiers bruts embarqués dans l'application.
BOM	Liste de versions compatibles pour toutes les bibliothèques Compose.
Composable	Fonction marquée `@Composable` qui décrit un morceau d'interface.
Coroutine	Tâche qui peut attendre (lecture de fichier…) sans bloquer l'écran.
data class	Classe Kotlin faite pour contenir des données, avec `copy()`.
dp / sp	Unités de taille indépendantes de l'écran ; sp suit en plus la taille de police.
État (state)	Ce que l'écran affiche à un instant donné.
JSON	Format texte de données : objets `{ }` et tableaux `[ ]`.
LazyColumn	Liste défilante qui ne dessine que les éléments visibles.
Modifier	Réglages d'un composable : taille, marges, fond, clic… L'ordre compte.
null	Absence de valeur. Un type avec `?` peut la contenir.
Recomposition	Compose rappelle les composables dont l'état a changé.
Repository	Classe chargée de fournir les données (ici, lire le JSON).
StateFlow	Flux qui contient une valeur et prévient de ses changements.
ViewModel	Garde l'état de l'écran ; survit à la rotation.